

Numerical Analysis – Lecture 1

1 LU factorization of matrices

1.1 Definition and applications

Let A be a real $n \times n$ matrix. We say that the $n \times n$ matrices L and U are an *LU factorization* of A if (1) L is lower-triangular (i.e., $L_{i,j} = 0$, $i < j$), (2) U is upper-triangular, $U_{i,j} = 0$, $i > j$, and (3) $A = LU$. Therefore the factorization takes the form

$$\begin{bmatrix} \square \end{bmatrix} = \begin{bmatrix} \square \end{bmatrix} \times \begin{bmatrix} \square \end{bmatrix}.$$

Application 1 Calculation of a determinant: $\det A = \det L \det U = \prod_{k=1}^n L_{k,k} \prod_{k=1}^n U_{k,k}$.

Application 2 Testing nonsingularity: $A = LU$ is nonsingular iff all the diagonal elements of L and U are nonzero.

Application 3 Solution of linear systems: Let $A = LU$ and suppose we wish to solve $A\mathbf{x} = \mathbf{b}$. This is the same as $L(U\mathbf{x}) = \mathbf{b}$, which we decompose into $L\mathbf{y} = \mathbf{b}$, $U\mathbf{x} = \mathbf{y}$. Both latter systems are triangular and can be calculated easily. Thus, $L_{1,1}y_1 = b_1$ gives y_1 , next $L_{2,1}y_1 + L_{2,2}y_2 = b_2$ yields y_2 etc. Having found $\mathbf{y}$, we solve for $\mathbf{x}$ by reversing the order: $U_{n,n}x_n = y_n$ gives x_n , $U_{n-1,n-1}x_{n-1} + U_{n-1,n}x_n = y_{n-1}$ produces x_{n-1} and so on.

Application 4 The inverse of A : It is straightforward to devise a direct way of calculating the inverse of triangular matrices, subsequently forming $A^{-1} = U^{-1}L^{-1}$.

1.2 The calculation of LU factorization

We denote the *columns* of L by $\mathbf{l}_1, \mathbf{l}_2, \dots, \mathbf{l}_n$ and the *rows* of U by $\mathbf{u}_1^T, \mathbf{u}_2^T, \dots, \mathbf{u}_n^T$. Hence

$$A = LU = \begin{bmatrix} \mathbf{l}_1 & \mathbf{l}_2 & \cdots & \mathbf{l}_n \end{bmatrix} \begin{bmatrix} \mathbf{u}_1^T \\ \mathbf{u}_2^T \\ \vdots \\ \mathbf{u}_n^T \end{bmatrix} = \sum_{k=1}^n \mathbf{l}_k \mathbf{u}_k^T. \quad (1.1)$$

Since the first $k-1$ components of $\mathbf{l}_k$ and $\mathbf{u}_k$ are all zero, each rank-one matrix $\mathbf{l}_k \mathbf{u}_k^T$ has zeros in its first k rows and columns.

Assume that the factorization exists and that A is nonsingular. Since $\mathbf{l}_k \mathbf{u}_k^T$ stays the same if we replace $\mathbf{l}_k \rightarrow \alpha \mathbf{l}_k$, $\mathbf{u}_k \rightarrow \alpha^{-1} \mathbf{u}_k$, where $\alpha \neq 0$, we may assume w.l.o.g. that all diagonal elements of L equal one. In other words, the k th row of $\mathbf{l}_k \mathbf{u}_k^T$ is $\mathbf{u}_k^T$ and its k th column is $U_{k,k}$ times $\mathbf{l}_k$.

We commence from $k = 1$. Since the first elements of $\mathbf{l}_k$ and $\mathbf{u}_k$ are zero for $k \geq 2$, it follows from (1.1) that $\mathbf{u}_1^T$ is the first row of A and $\mathbf{l}_1$ is the first column of A , divided by $A_{1,1}$ (so that $L_{1,1} = 1!$).

Having found $\mathbf{l}_1$ and $\mathbf{u}_1$, we form the matrix $A - \mathbf{l}_1 \mathbf{u}_1^T = \sum_{k=2}^n \mathbf{l}_k \mathbf{u}_k^T$. The first row & column of A are zero and it follows that $\mathbf{u}_2^T$ is the second row of $A - \mathbf{l}_1 \mathbf{u}_1^T$, while $\mathbf{l}_2$ is its second column, scaled so that $L_{2,2} = 1$.

The LU algorithm: Set $A_0 := A$. For all $k = 1, 2, \dots, n$ set $\mathbf{u}_k^T$ to the k th row of A_{k-1} and $\mathbf{l}_k$ to the k th column of A_{k-1} , scaled so that $L_{k,k} = 1$. Further, calculate $A_k := A_{k-1} - \mathbf{l}_k \mathbf{u}_k^T$ before incrementing k .

Note that all elements in the first k rows & columns of A_k are zero. Hence, we can use the storage of the original A to accumulate L and U .

1.3 Relation to Gaussian elimination

The equation $A_k = A_{k-1} - \mathbf{l}_k \mathbf{u}_k^T$ has the property that the j th row of A_k is the j th row of A_{k-1} minus $L_{j,k}$ times $\mathbf{u}_k^T$ (the k th row of A_{k-1}). Moreover, the multipliers $L_{k,k}, L_{k+1,k}, \dots, L_{n,k}$ are chosen so that the outcome of this *elementary row operation* is that the k th column of A_k is zero. This construction is analogous to Gaussian elimination for solving $A\mathbf{x} = \mathbf{b}$. An important difference is that in LU we do not consider the right hand side $\mathbf{b}$ until the factorization is complete. This is useful e.g. when there are many right hand sides, in particular if not all the $\mathbf{b}$'s are known at the outset.

1.4 Pivoting

Naive LU factorization fails when, for example, $A_{1,1} = 0$. The remedy is to exchange rows of A , a technique called *column pivoting*. This is equivalent to picking a suitable equation for eliminating the first unknown in Gaussian elimination. Specifically, column pivoting means that, having obtained A_{k-1} , we exchange two rows of A_{k-1} so that the element of largest magnitude in the k th column is in the 'pivotal position' (k, k) . In other words,

$$|(A_{k-1})_{k,k}| = \max\{|(A_{k-1})_{j,k}| : j = 1, 2, \dots, n\}.$$

Of course, the same exchange is required in the part of L that has been formed already (i.e., the first $k - 1$ columns). Also, we need to record the permutation of rows to solve for the right hand side and/or to compute the determinant.

This procedure copes with zeros at the pivot position, except when the whole k th column of A_k is zero – in that case it is usual to let $\mathbf{l}_k$ be the k th unit vector (and, as before, choose $\mathbf{u}_k^T$ as the k th row of A_k).

An important advantage of this procedure is that every element of L has magnitude at most one. This avoids not just division by zero but also tends to reduce the chance of very large numbers occurring during the factorization, a phenomenon that might lead to accumulation of *roundoff error* and to *ill conditioning*.

In *row pivoting* one exchanges columns of A_{k-1} , rather than rows (sic!), whereas *total pivoting* corresponds to exchange of both rows and columns, so that the modulus of the pivotal element $(A_{k-1})_{k,k}$ is maximized.



This and subsequent handouts will be available at the WWW site
<http://www.damtp.cam.ac.uk/DAMTP/user/na/PartIB/Handouts.html>